



OPERATING SYSTEMS **Security**



The Security Problem

- System secure if resources used and accessed as intended under all circumstances.
 - ◆ Unachievable.
- Intruders (crackers) attempt to breach security.
- Threat is potential security violation.
- Attack is attempt to breach security.
- Attack can be accidental or malicious.
- Easier to protect against accidental than malicious misuse.

Security Violation Categories

- **Breach of confidentiality:** Unauthorized reading of data.
- **Breach of integrity:** Unauthorized modification of data.
- **Breach of availability:** Unauthorized destruction of data.
- **Theft of service:** Unauthorized use of resources.
- **Denial of service (DOS):** Prevention of legitimate use.

Security Violation Methods

- **Masquerading (breach authentication):** Pretending to be an authorized user to escalate privileges.
- **Replay attack:** As is or with message modification.
- **Man-in-the-middle attack:** Intruder sits in data flow, masquerading as sender to receiver and vice versa.
- **Session hijacking:** Intercept an already-established session to bypass authentication.
- **Privilege escalation:** Common attack type with access beyond what a user or resource is supposed to have.

Security Measure Levels

- Impossible to have absolute security, but make cost to perpetrator sufficiently high to deter most intruders.
- Security must occur at four levels to be effective:
 - ◆ **Physical:** Data centers, servers, connected terminals.
 - ◆ **Application:** Malicious apps can cause security problems.
 - ◆ **Operating System:** Protection mechanisms, debugging.
 - ◆ **Network:** Intercepted communications, interruption, DOS.
- Security is as weak as the weakest link in the chain.
- Humans a risk too via phishing and social-engineering attacks.

Program Threats

- Many variations, many names.
- **Trojan Horse:**
 - ◆ Code segment that misuses its environment.
 - ◆ Exploits mechanisms for allowing programs written by users to be executed by other users.
 - ◆ Spyware, pop-up browser windows, covert channels.
 - ◆ Up to 80% of spam delivered by spyware-infected systems.
- **Trap Door:**
 - ◆ Specific user identifier or password that circumvents normal security procedures.
 - ◆ Could be included in a compiler.

Program Threats

- **Malware** - Software designed to exploit, disable, or damage computer.
- **Trojan Horse** – Program that acts in a clandestine manner.
 - ♦ **Spyware** – Program frequently installed with legitimate software to display ads, capture user data.
 - ♦ **Ransomware** – locks up data via encryption, demanding payment to unlock it
- Others include trap doors, logic bombs.
- All try to violate the Principle of Least Privilege.

THE PRINCIPLE OF LEAST PRIVILEGE

“The principle of least privilege. Every program and every privileged user of the system should operate using the least amount of privilege necessary to complete the job. The purpose of this principle is to reduce the number of potential interactions among privileged programs to the minimum necessary to operate correctly, so that one may develop confidence that unintentional, unwanted, or improper uses of privilege do not occur.”—Jerome H. Saltzer, describing a design principle of the Multics operating system in 1974: <https://pdfs.semanticscholar.org/1c8d/06510ad449ad24fbdd164f8008cc730cab47.pdf>.

- Goal frequently is to leave behind **Remote Access Tool (RAT)** for repeated access.

Program Threats

- C Program with **Buffer-overflow** Condition:

```
#include <stdio.h>
#define BUFFER SIZE 256
int main(int argc, char *argv[])
{
    char buffer[BUFFER SIZE];
    if (argc < 2)
        return -1;
    else {
        strcpy(buffer, argv[1]);
        return 0;
    }
}
```

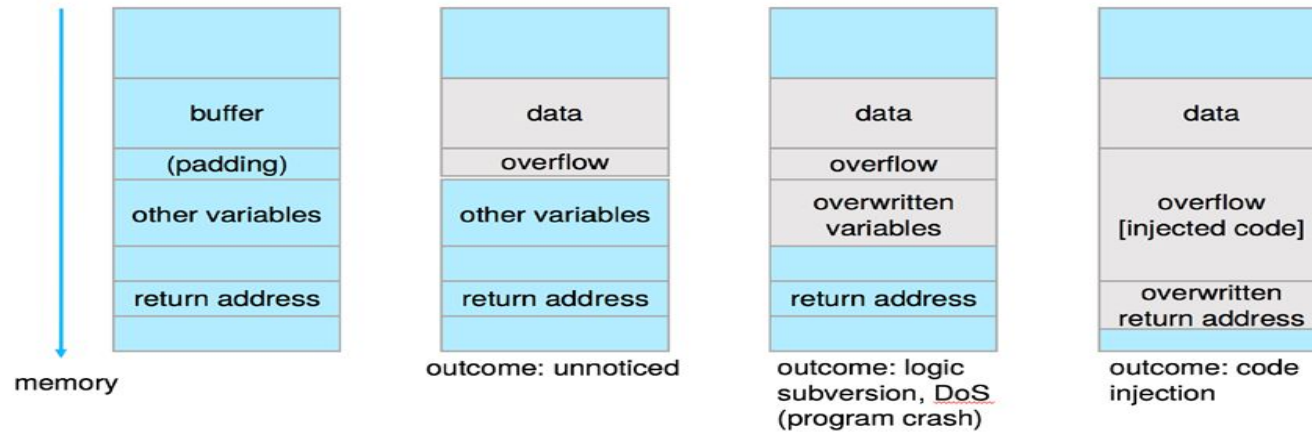
➔ **Code review** can help – programmers review each other's code, looking for logic flows, programming flaws.

Program Threats

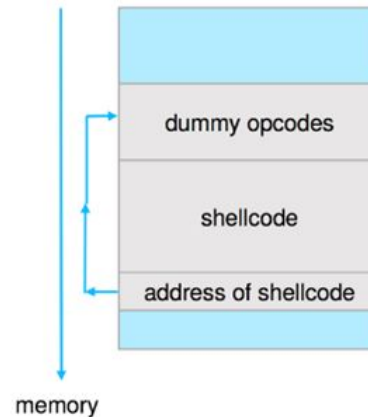
- **Code-injection attack** occurs when system code is not malicious but has bugs allowing executable code to be added or modified.
 - ◆ Results from poor or insecure programming paradigms, commonly in low level languages like C or C++ which allow for direct memory access through pointers.
 - ◆ Goal is a buffer overflow in which code is placed in a buffer and execution caused by the attack.
 - ◆ Can be run by **script kiddies** – use tools written but exploit identifiers.

Program Threats

- Outcomes from **code injection**



- Frequently use trampoline to code execution to exploit buffer overflow:



Program Threats

→ Viruses:

- ◆ Code fragment embedded in legitimate program.
- ◆ Self-replicating, designed to infect other computers.
- ◆ Very specific to CPU architecture, operating system, applications.
- ◆ Usually borne via email or as a macro.
- ◆ Visual Basic Macro to reformat hard drive.

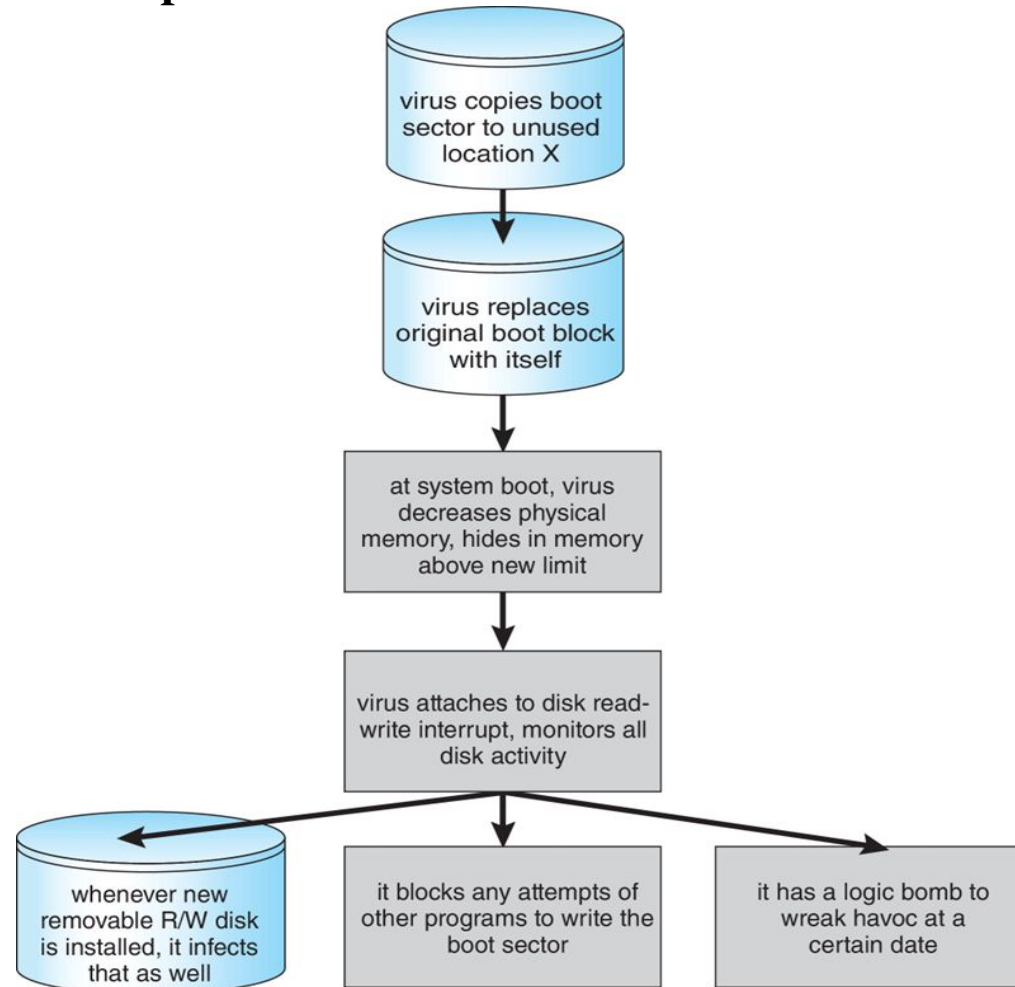
```
Sub AutoOpen()  
Dim oFS  
    Set oFS = CreateObject('Scripting.FileSystemObject')  
    vs = Shell('c:command.com /k format c:', vbHide)  
End Sub
```

Program Threats

- ➔ **Virus dropper** inserts virus onto the system.
- ➔ Many categories of viruses, literally many thousands of viruses:
 - ◆ File / parasitic
 - ◆ Boot / memory
 - ◆ Macro
 - ◆ Source code
 - ◆ Polymorphic to avoid having a virus signature
 - ◆ Encrypted
 - ◆ Stealth
 - ◆ Tunneling
 - ◆ Multipartite
 - ◆ Armored

Program Threats

A Boot-sector Computer Virus



Program Threats

→ **Worms** – use spawn mechanism; standalone program.

→ **Internet worm:**

- ◆ Exploited UNIX networking features (remote access) and bugs in finger and sendmail programs.
- ◆ Exploited trust-relationship mechanism used by rsh to access friendly systems without use of password.
- ◆ Grappling hook program uploaded main worm program.
- ◆ 99 lines of C code .
- ◆ Hooked system then uploaded main code, tried to attack connected systems.
- ◆ Also tried to break into other users accounts on local system via password guessing.
- ◆ If target system already infected, abort, except for every 7th time.

Four-layered Model of Security

types of attacks

logic bugs, design flaws, code injections
insecure defaults, platform vulnerabilities
sniffing, spoofing, masquerading
console access, hardware-based attacks



application

operating system

network

physical

attack prevention methods



sandboxing, software restrictions

patches, reconfiguration, hardening

encryption, authentication, filtering

guards, vaults, device data encryption

Implementing Security Defences

- Defence in depth is most common security theory – multiple layers of security.
- Security policy describes what is being secured.
- Vulnerability assessment compares real state of system / network compared to security policy.
- Intrusion detection endeavors to detect attempted or successful intrusions.
 - ◆ Signature-based detection spots known bad patterns.
 - ◆ Anomaly detection spots differences from normal behavior.
 - ◆ Can detect zero-day attacks.
 - ◆ False-positives and false-negatives a problem.
- Virus protection:
 - ◆ Searching all programs or programs at execution for known virus patterns.
 - ◆ Or run in sandbox so can't damage system.
- Auditing, accounting, and logging of all or specific system or network activities.
- Practice safe computing – avoid sources of infection, download from only “good” sites, etc.

Firewall to Protect Systems and Networks

- A network firewall is placed between trusted and untrusted hosts.
 - ◆ The firewall limits network access between these two security domains.
- Can be tunneled or spoofed.
 - ◆ Tunneling allows disallowed protocol to travel within allowed protocol (i.e., telnet inside of HTTP).
 - ◆ Firewall rules typically based on hostname or IP address which can be spoofed.
- Personal firewall is software layer on given host.
 - ◆ Can monitor / limit traffic to and from the host.
- Application proxy firewall understands application protocol and can control them (i.e., SMTP).
- System-call firewall monitors all important system calls and apply rules to them (i.e., this program can execute that system call).

Network Security Through Domain Separation Via Firewall

